CORECONF vs NETCONF / RESTCONF: On-the-Wire Byte Savings

This analysis quantifies the data-transfer savings of **CORECONF** (`draft-ietf-core-comi`, CBOR-over-CoAP with numeric SIDs) compared to **NETCONF** (XML-over-SSH, RFC 6241) and **RESTCONF** (JSON-over-HTTP, RFC 8040).

All figures use the exact same operation from this repo's example: turning the coffee maker on with `power=true, cups=2, strength=strong`, against the `example-coffee-maker` YANG model.

The CBOR payload sizes are measured directly from the running example; the HTTP/SSH framing figures are representative (typical header sizes) rather than captured from a live socket.

Application payload (the config edit itself)

Encoding	Protocol	Bytes	vs CORECONF
CBOR + SIDs	CORECONF (iPATCH)	17	1x
JSON	RESTCONF PATCH body	81	~4.8x
XML	NETCONF `<edit-config>`	378	~22x

Minimal "just flip power" case:

Encoding	Bytes	Wire representation
CBOR + SIDs	5	`a1 19 ea 6b f5` = `{60011: true}`
JSON	~50	`{"example-coffee-maker:power":true}`
XML	~250	`<edit-config>...<power>true</power>`

Why the savings

The reduction comes from three compounding factors:

1. Numeric SIDs instead of identifier strings

`power` becomes SID `60011`, encoded in 3 bytes (`19 ea 6b`), versus `"power"` (7 bytes in JSON) or `<power>...</power>` (16 bytes in XML). Namespace URIs cost **0 bytes** on the wire in CORECONF because the module is implied by the SID range — no `urn:example:coffee-maker` is transmitted.

2. Binary CBOR instead of text

- `true` = 1 byte (`0xf5`)
- integer `2` = 1 byte
- enum `strong` maps to integer `2` (1 byte) instead of a 6-byte string
- no quotes, colons, braces, or closing tags

3. CoAP framing instead of HTTP / SSH

This dominates the *total* transaction savings:

- **vs NETCONF:** runs over SSH, requiring TCP + SSH handshakes and a `<hello>` capability exchange (~269 bytes) before the first edit. The edit RPC itself is 378 bytes of XML. CoAP has no session setup — a request is a single ~20-40 byte datagram plus the 17-byte payload.
- **vs RESTCONF:** the JSON body is 81 bytes, but the full HTTP request with request line and headers is ~272 bytes. The equivalent CoAP message replaces those headers with a few compact binary options (URI-path `/c`, content-format as a 1-byte integer), so the whole datagram is ~35-45 bytes total.

Rough total-transaction picture

	NETCONF	RESTCONF	CORECONF
Transport setup	TCP + SSH handshake + `<hello>` (~269 B)	TCP (+ TLS handshake)	none (UDP, 0-RTT)
Per-request framing	XML RPC envelope	HTTP request line + headers (~190 B)	CoAP binary options (~15-25 B)
Config payload	378 B (XML)	81 B (JSON)	17 B (CBOR+SID)
Round trips to 1st edit	many (TCP + SSH + hello)	2-4 (TCP + TLS)	1

Summary

- The **payload alone** is roughly **5x smaller than RESTCONF/JSON** and **~20x smaller than NETCONF/XML**.
- Including transport framing and the elimination of session setup, a complete CORECONF transaction is commonly **an order of magnitude smaller in total bytes** and uses **far fewer round trips**.
- This is why CORECONF targets constrained IoT devices and lossy low-power networks, where NETCONF and RESTCONF are impractical.

Caveats

- Exact CoAP header/option size varies with token length and URI.
- HTTP/SSH figures are representative, not measured off a live socket.
- TLS/DTLS and OSCORE overhead are not included. OSCORE adds a small fixed AEAD tag plus one CoAP option — still far lighter than a TLS record.
- CBOR payload sizes (17 B, 5 B) are measured directly from the running example in this repository (`coffee-example/`).